

Explicación Técnica: SPEC 01 - Baseline reproducible de repositorios, firmware y hardware

- **Ruta del SPEC:** `specs/01-baseline-reproducible-repositorios-firmware-hardware.md`
- **Estado:** Approved
- **Nivel de explicación:** Técnico (Technical)
- **Fecha de generación:** 2026-07-20

[!NOTE]

Esta explicación constituye un desglose técnico y operativo detallado de las condiciones descritas en el SPEC 01. El documento original de la especificación es el contrato normativo oficial y el que prevalece sobre este análisis en caso de cualquier discrepancia.

1. En un minuto

- **Problema:** El proyecto `clawd-meter` (firmware de Cheap Yellow Display) y `ai-usage-meter` (repositorio coordinador) carecen de una línea base controlada. Las dependencias del toolchain son flotantes, no hay una estructura formal de submódulo Git, el hardware físico CYD opera bajo firmware no certificado y existe riesgo de fuga o persistencia de secretos (como la `sessionKey` de Claude) tanto en el repositorio como en los sectores de almacenamiento de la flash de 4 MB.
- **Resultado propuesto:** Establecer un baseline de software determinista en macOS `arm64` (fijando Python `3.14.6`, PlatformIO `6.1.19` y subpaquetes del compilador via hashes), reparar la topología Git utilizando submódulos con URLs HTTPS públicas/SSH privadas, y certificar el hardware físico mediante un backup completo cifrado, borrado total de flash y carga limpia probada en una red Wi-Fi aislada.
- **Estado actual y permisos:** El SPEC 01 está **Approved**. Este estado autoriza al agente a iniciar la fase de ejecución técnica mediante el comando `/spec-impl`.
- **Beneficiarios:** Desarrolladores (entorno de compilación local robusto y reproducible) y el usuario final (seguridad de credenciales y estabilidad operativa del dispositivo).

2. Qué cambia y qué no

Cambios en el alcance (Consecuencias observadas)

- **Topología de Repositorios:** La carpeta `clawd-meter/` se formaliza como submódulo Git mediante el archivo `.gitmodules`, rastreada en el índice con el modo `160000`. Se reconfiguran los remotos Git separando las URLs de lectura pública (`https://`) y escritura (`git@github.com:...`).
- **Toolchain de Compilación:** Aislamiento del entorno mediante un script de bootstrap (`scripts/bootstrap-tools.sh`) que crea `.venv-tools` y restringe la instalación mediante hashes deterministas (`--require-hashes`). La compilación de PlatformIO se fuerza a utilizar un `PLATFORMIO_CORE_DIR` aislado para no contaminarse de dependencias heredadas de `~/platformio`.
- **Seguridad de Secretos:** Integración de Gitleaks local en `.tools/gitleaks` versión `8.30.1` para escanear todo el historial de Git y árboles de trabajo.
- **Hardware y Persistencia:** Extracción y cifrado local en reposo de la imagen de flash de 4 MB (`4194304` bytes) antes de ejecutar un borrado total (`erase_flash`). Flashing limpio de las 4 regiones (bootloader, particiones, app y LittleFS) a través de un puerto serie explícito.

Fuera de alcance (Permanecen intactos)

- **Código Fuente C++ / UI:** No se modifica la lógica de la aplicación del firmware ni la interfaz web de LittleFS. Ninguna funcionalidad relacionada con el Bridge o endpoints de Codex/ChatGPT Plus entra en este SPEC.

- **Versión del Firmware:** La constante **FW_VERSION** en el código fuente no se modifica; la identidad de la línea base se mapea a los commits y hashes de los bloqueos generados.

- **Compatibilidad de Plataformas:** No se proporciona soporte ni validación para compilación en hosts Linux, Windows o macOS basados en Intel. Tampoco se valida físicamente el hardware ESP8266 (**nodemcu v2**).

3. Antes y después

Dimensión	Estado Anterior (Observado / Inferido)	Estado Especificado (Garantizado)
Relación de Git	Carpeta 'clawd-meter/' desvinculada, versionada directamente o con submódulo roto sin '.gitmodules'.	Submódulo formalizado con gitlink de 40 caracteres, detached HEAD alineado en clon anónimo y asimetría de push/pull configurada.
Reproducibilidad	Compilación susceptible a la actualización de paquetes locales y variables globales ('python', 'pio').	Entorno sellado en '.venv-tools' con Python '3.14.6' y paquetes fijados por hash en 'requirements-tools.lock'.
PlatformIO	Descarga de paquetes del framework Arduino y toolchains Xtensa flotantes basados en la caché global.	Registro inmutable de dependencias internas en 'toolchain/platformio-packages.lock.json' y compilación en directorio aislado.
Secretos en Git	Posible exposición inadvertida de la cookie 'sessionKey' o tokens locales en el historial o LittleFS.	Redacción total, auditoría con Gitleaks y política obligatoria de revocación inmediata si se detecta una credencial activa.
Estado del CYD	Dispositivo con sectores de flash residuales y variables de WiFi y API keys previas activas.	Flash borrada por completo, instalación de origen 100% conocido y prueba de arranque aislada sin credenciales iniciales.

4. Cómo encajan las piezas

El flujo de compilación y flashing opera con interfaces aisladas:

```
[Repositorio ai-usage-meter]
|
|---> [Submódulo Git: clawd-meter (firmware)]
|
|      \---> src/core/config.h (Target: CYD / ESP32)
```

```

|
|---> [.venv-tools (Python 3.14.6)]
|
|      |---> uv 0.11.29 / pip 26.1.2
|
|      |---> Gitleaks 8.30.1 --> Escaneo de secretos
|
|      \---> PlatformIO 6.1.19
|
|          |
|          |--- [PLATFORMIO_CORE_DIR Temporal]
|          |
|          |      \--- Resolutor de platform_packages
|          |
|          |
|          \--- [Build Artifacts]
|
|                  |--- bootloader.bin (0x1000)
|                  |--- partitions.bin (0x8000)
|                  |--- firmware.bin   (0x10000)
|                  \--- littlefs.bin   (0x290000)
|
|                      |
|                      v (esptool.py @ puerto autorizado)
[Hardware CYD (ESP32)] <-----/
|
|---> [Backup Cifrado (AES-256)] --> Library/Application Support/...
\---> [Checklist Funcional (10 min)] --> Conectado a Red Temporal Aislada

---
```

5. Paso a paso

1. Preflight sin modificaciones

- **Acción:** Inspección pasiva del estado de Git local en ambos repositorios, verificación de remotos y validación de existencia del repositorio remoto en GitHub.
- **Justificación:** Previene la pérdida de referencias Git locales huérfanas antes de alterar el submódulo.
- **Archivos afectados:** `ai-usage-meter` y `clawd-meter` (árbol de Git).
- **Dependencias:** Ninguna.
- **Verificación:** Ejecución de comandos de inspección y registro del baseline de commits.
- **Riesgo / Impacto:** Nulo, no realiza escrituras.
- **Intrincación de la implementación:** Baja.
- **Capacidad de razonamiento recomendada:** Baja.

2. Higiene de repositorio, locks y bootstrap

- **Acción:** Renombrado seguro de `Docs/` a `docs/` en sistemas case-insensitive. Configuración de `.gitignore` raíz y construcción de `requirements-tools.lock` instalando Python 3.14.6, uv 0.11.29 y pip 26.1.2. Descarga local y validación de Gitleaks 8.30.1.
- **Justificación:** Garantiza que el entorno local se construya a partir de binarios verificados por firma criptográfica y sin interferencias del entorno del sistema.
- **Archivos afectados:** `.gitignore`, `requirements-tools.lock`, `toolchain/bootstrap-checksums.txt`, `scripts/bootstrap-tools.sh`.
- **Dependencias:** Paso 1.

- **Verificación:** Validación de rutas y versiones de ejecutables en `.venv-tools/bin/`.
- **Riesgo / Impacto:** Bajo.
- **Intrincación de la implementación:** Media.
- **Capacidad de razonamiento recomendada:** Media.

3. Gate inicial de secretos

- **Acción:** Auditoría de todo el historial de commits y archivos de trabajo utilizando Gitleaks y búsquedas dirigidas para variables sensibles como `sessionKey`, `claudeKey` y `apiToken`.
- **Justificación:** Asegura que no se publique ningún secreto antes de empujar el código del baseline a repositorios remotos.
- **Archivos afectados:** Todo el árbol de archivos.
- **Dependencias:** Paso 2.
- **Verificación:** Reporte con 0 detecciones de secretos no resueltos.
- **Riesgo / Impacto:** Alto en caso de falso negativo (exposición de claves del usuario).
- **Intrincación de la implementación:** Media.
- **Capacidad de razonamiento recomendada:** Alto.

4. Clasificación y reparación de la topología Git

- **Acción:** Vinculación formal de `clawd-meter/` como submódulo, configurando `.gitmodules` sin rastreo de rama y con URLs HTTPS para lectura, y URLs SSH específicas para push en el entorno de desarrollo.
- **Justificación:** Permite clonaciones limpias y anónimas de manera externa, mientras retiene la capacidad de escritura para el mantenedor.
- **Archivos afectados:** `.gitmodules`, índice Git.
- **Dependencias:** Pasos 1 y 3.
- **Verificación:** Clonación de prueba limpia en directorio temporal con detached HEAD exacto en el gitlink.
- **Riesgo / Impacto:** Alto. Una mala manipulación de submódulos puede causar pérdida de commits locales no sincronizados en `clawd-meter/`.
- **Intrincación de la implementación:** Alta.
- **Capacidad de razonamiento recomendada:** Alto.

5. Fijación de PlatformIO y builds reproducibles

- **Acción:** Fijación estricta de bibliotecas de dependencias del firmware (`TFT_eSPI`, `ArduinoJson`, `XPT2046`) en `platformio.ini` y generación de `toolchain/platformio-packages.lock.json`. Compilación de prueba limpia en un `PLATFORMIO_CORE_DIR` aislado.
- **Justificación:** Elimina compilaciones flotantes debidas a actualizaciones no deseadas de bibliotecas o herramientas de PlatformIO.
- **Archivos afectados:** `clawd-meter/platformio.ini`, `toolchain/platformio-packages.lock.json`, `toolchain/build-warnings-allowlist.yml`.
- **Dependencias:** Paso 2.
- **Verificación:** Compilación exitosa (`exit 0`) con cero warnings no declarados en el allowlist.
- **Riesgo / Impacto:** Medio. Problemas de incompatibilidad de compilador o dependencias cruzadas.
- **Intrincación de la implementación:** Alta.
- **Capacidad de razonamiento recomendada:** Alto.

6. Publicación preliminar del coordinador y test de clonación limpia

- **Acción:** Push a la rama `main` de `ai-usage-meter` y clonación de prueba recursiva en un directorio temporal limpio desde un origen vacío.
- **Justificación:** Valida de manera empírica que un tercero puede obtener la misma base funcional a partir de repositorios públicos.
- **Archivos afectados:** Repositorio coordinador remoto.
- **Dependencias:** Pasos 4 y 5.
- **Verificación:** Bootstrap y compilación de firmware/LittleFS exitosas en la clonación temporal.
- **Riesgo / Impacto:** Bajo.
- **Intrincación de la implementación:** Media.
- **Capacidad de razonamiento recomendada:** Alto.

7. Identificación del CYD y backup sensible

- **Acción:** Identificación del chip físico ESP32 y descarga de sus 4 MB de flash (`0x400000` bytes) a un archivo `.bin` en un directorio local seguro con permisos restrictivos (`0700/0600`) y cifrado AES-256.
- **Justificación:** Preservación del estado anterior para investigación forense de fallos, previniendo la pérdida irreversible de calibraciones físicas.
- **Archivos afectados:** Local storage host (`~/Library/Application Support/AIUsageMeter/backups/`).
- **Dependencias:** Paso 2 (esptool en entorno virtual).
- **Verificación:** Validación del SHA-256 de la imagen descargada de 4194304 bytes.
- **Riesgo / Impacto:** Alto. Manejo de información privada contenida en el backup del usuario.
- **Intrincación de la implementación:** Alta.
- **Capacidad de razonamiento recomendada:** Alto.

8. Borrado físico y flashing limpio de firmware

- **Acción:** Borrado completo de la memoria flash (`erase_flash`) del chip y posterior flashing de los artefactos producidos (bootloader, partition-table, firmware, littlefs) en sus offsets de memoria correspondientes.
- **Justificación:** Garantiza que no queden datos residuales ni configuraciones corruptas en el hardware.
- **Archivos afectados:** Dispositivo físico CYD.
- **Dependencias:** Pasos 5 y 7.
- **Verificación:** Códigos de retorno de flashing exitosos y validación de hash de escritura (`verify_image`).
- **Riesgo / Impacto:** Alto. Posibilidad de sobrescribir otro dispositivo o de interrumpir el flashing causando un estado inoperable (unbricking requerido).
- **Intrincación de la implementación:** Alta.
- **Capacidad de razonamiento recomendada:** Alto.

9. Checklist funcional y persistencia

- **Acción:** Protocolo de prueba manual del dispositivo en una red de invitados aislada, configurando datos de prueba sin secretos de la API de Claude.
- **Justificación:** Asegura la operatividad de los periféricos físicos (pantalla, backlight, chip táctil XPT2046) y la estabilidad de la pila de memoria (heap).
- **Archivos afectados:** Registro `docs/hardware-test-checklist.md`.
- **Dependencias:** Paso 8.
- **Verificación:** Ejecución de 12 pruebas físicas durante una ventana de monitoreo de 10 minutos (NTP, rotación, táctil, heap sin fugas).
- **Riesgo / Impacto:** Medio.
- **Intrincación de la implementación:** Media.
- **Capacidad de razonamiento recomendada:** Media.

10. Sincronización final de gitlink y cierre

- **Acción:** Push del firmware reparado y actualización del puntero gitlink en el coordinador apuntando al commit final.
- **Justificación:** Cierra el lazo asegurando que el repositorio coordinador apunta a la versión exacta del submódulo que acaba de aprobarse.
- **Archivos afectados:** Gitlink en `ai-usage-meter`.
- **Dependencias:** Pasos 6 y 9.
- **Verificación:** Coincidencia exacta de SHA entre el índice del coordinador y el HEAD de `clawd-meter`.
- **Riesgo / Impacto:** Alto. La falta de atomicidad de commits entre repositorios puede romper compilaciones de terceros.
- **Intrincación de la implementación:** Alta.
- **Capacidad de razonamiento recomendada:** Alto.

11. Evidencias documentales y gate de secretos final

- **Acción:** Completar registros y eliminar temporales creados. Validar la redacción final de secretos en reportes públicos.
- **Justificación:** Previene fugas accidentales a través de logs de depuración o capturas asociadas a las pruebas.
- **Archivos afectados:** `docs/reproducibility-baseline.md`, `docs/hardware-test-checklist.md`.
- **Dependencias:** Pasos 9 y 10.
- **Verificación:** Archivos de documentación comiteados en limpio y sin rastros de SSID, claves,

ni rutas de directorios locales personales.

- **Riesgo / Impacto:** Medio. Fugas de metadatos o información del entorno.
- **Intrincación de la implementación:** Media.
- **Capacidad de razonamiento recomendada:** Alto.

12. Validación de compilación desde cero (Limpio)

- **Acción:** Proceso final de clonación del coordinador recursivo en entorno limpio, aislamiento de cachés de PlatformIO y Python, ejecución de bootstrap y compilación.
- **Justificación:** Prueba definitiva de reproducibilidad autónoma del proyecto.
- **Archivos afectados:** Directorios de pruebas temporales.
- **Dependencias:** Paso 11.
- **Verificación:** Compilación exitosa idéntica del firmware e imagen LittleFS.
- **Riesgo / Impacto:** Bajo.
- **Intrincación de la implementación:** Media.
- **Capacidad de razonamiento recomendada:** Media.

6. Decisiones y compensaciones

- **Fases Desacopladas (Fase A vs Fase B):**
- **Elegido:** Permitir la compleción de la Fase A (repositorio, seguridad y builds reproducibles) incluso en ausencia del hardware CYD.
- **Rechazado:** Exigir que todo el SPEC dependa de la presencia física del dispositivo.
- **Compensación:** El software y el toolchain pueden validarse y versionarse inmediatamente en el pipeline de desarrollo, posponiendo la Fase B de forma aislada sin detener el progreso lógico del equipo.
- **Aislamiento de PlatformIO (PLATFORMIO_CORE_DIR):**
- **Elegido:** Compilar usando un directorio aislado para la descarga de toolchains de Arduino y esptool.
- **Rechazado:** Depender del directorio global `~/.platformio` por defecto.
- **Compensación:** La compilación tarda unos minutos más al tener que inicializar el entorno aislado en la primera ejecución, pero garantiza al 100% que no hay paquetes residuales o "contaminados" de otros proyectos en la máquina del desarrollador.
- **Backup Forense:**
- **Elegido:** El backup de 4 MB de flash se trata como un elemento altamente sensible (cifrado en reposo y con destrucción obligatoria a los 30 días).
- **Rechazado:** Utilizar el backup como un rollback tradicional sencillo y sin controles.
- **Compensación:** Si el firmware de fábrica tuviera credenciales activas del usuario expuestas en la flash, restaurar la copia ciegamente anularía el proceso de saneamiento de secretos. El backup se restringe exclusivamente para recuperar el hardware en caso de un fallo físico catastrófico durante el flashing.

7. Dónde debe participar el usuario

El usuario o mantenedor principal debe tomar decisiones clave en:

- **Política de remediación ante secretos históricos:** Si Gitleaks detecta que se publicaron credenciales de Claude en commits antiguos en el repositorio remoto, el usuario debe decidir entre reescribir de manera destructiva el historial de Git (afecta a otros colaboradores) o revocar la credencial de forma activa en la plataforma de Claude y registrarla como revocada en el SPEC.
- **Puerto Serial Autorizado:** Durante los pasos de backup y flashing, el usuario debe identificar explícitamente el nodo de puerto de su máquina macOS (ej. `/dev/cu.usbserial-1410`) para prevenir que las herramientas actúen sobre otros microcontroladores conectados.
- **Checklist Funcional Manual (Fase B):** El usuario debe ejecutar el test físico del dispositivo conectado a una red WiFi segura aislada y completar las observaciones humanas del LCD, la respuesta táctil de 10 toques consecutivos y verificar la lectura del tamaño de memoria dinámica (heap) mediante la consola serial.

8. Riesgos y comportamiento degradado

- **Fallo en NTP o Servicio Meteorológico (Open-Meteo):**
- **Efecto observable:** El reloj del CYD muestra una hora incorrecta o la pantalla de clima falla tras pasar los timeouts (60s NTP / 90s clima).
- **Mitigación especificada:** El test no fallará inmediatamente; se valida que el firmware maneje el estado degradado de manera segura (mostrando pantallas de error estables) y que se demuestre que la causa es de conectividad externa y no un fallo de código.
- **Inyección de credenciales antiguas desde el backup:**
- **Efecto observable:** Filtración de secretos tras una recuperación de hardware.
- **Mitigación especificada:** Prohibición absoluta de reescribir la flash utilizando el backup de 4 MB sin una autorización por separado y una purga manual de credenciales documentada.
- **Fuga de credenciales en capturas de pantalla o fotografías de verificación:**
- **Efecto observable:** Publicación accidental de credenciales activas o contraseñas WiFi del desarrollador.
- **Mitigación especificada:** Las evidencias visuales del hardware se sanitizan eliminando metadatos EXIF, se hashean y se restringe su almacenamiento en repositorios principales.

9. Cómo se demostrará la finalización

La finalización exitosa de la implementación del SPEC 01 se verificará mediante:

- **Evidencias de Integridad en Git:**
- Archivo `.gitmodules` creado con URL HTTPS pública y sin marcas de branch tracking flotante.
- `git submodule status` comenzando con un espacio limpio (sin prefijos `+` o `-`), indicando sincronía exacta entre el gitlink del coordinador y el HEAD del submódulo.
- **Determinismo del Entorno:**
- Archivos de bloqueo `requirements-tools.lock` y `toolchain/platformio-packages.lock.json` válidos.
- Ejecución exitosa de `scripts/bootstrap-tools.sh` en un macOS `arm64` limpio, verificando versiones exactas (Python `3.14.6`, PlatformIO `6.1.19`, uv `0.11.29`).
- **Flashing y Firma de Hardware:**
- Código de salida `0` tras ejecutar `esptool.py verify_image` sobre las regiones de bootloader, partitions, app y LittleFS.
- Existencia del archivo `docs/hardware-test-checklist.md` completo, con las firmas y el log redactado de las 12 pruebas físicas de la Fase B en estado `PASS`.
- **Verificación Repetible:**
- Ejecución exitosa y repetible del build de firmware en un clon de repositorio anónimo aislado.

10. Glosario

- **Gitlink:** Puntero interno de commit que Git utiliza en un repositorio padre para hacer referencia a una revisión fija y exacta de un submódulo (se visualiza en el índice de Git como permisos tipo `160000`).
- **Toolchain:** Conjunto de herramientas de software (compilador, enlazador, empaquetador) configuradas en una cadena secuencial para traducir el código C++ en binarios ejecutables para el ESP32.
- **esptool:** La utilidad basada en Python distribuida por Espressif para comunicarse con el cargador de arranque integrado en la memoria ROM del ESP32 a través del puerto serie.
- **LittleFS:** Un sistema de archivos liviano optimizado para microcontroladores y memoria flash SPI, utilizado en el CYD para almacenar recursos estáticos (HTML/CSS) y la configuración del usuario en `/config.json`.
- **Detached HEAD:** Estado en Git en el que la copia de trabajo de un repositorio apunta a un commit específico en lugar de a una rama activa (comportamiento estándar y requerido para submódulos compilados en producción).

11. Gaps y preguntas

- **Remediación en repositorios públicos actuales:** El SPEC define la bifurcación de estrategias entre historial no publicado e historial público de Git. Sin embargo, no se especifica si la rama remota actual de **ai-usage-meter** ya ha sido indexada por buscadores. *Recomendación:* Se debe asumir el peor de los casos y revocar activamente todas las credenciales asociadas antes de realizar el primer push del baseline, independientemente de la reescritura del historial.